

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия,
Системное и прикладное программное обеспечение

Дисциплина: Вычислительная математика

Лабораторная работа №4

Аппроксимация функции методом наименьших квадратов
Вариант №5

Выполнил:
Карнажицкий Максим Романович
Группа: Р3211

Проверил:
Малышева Т. А.

Содержание

1. Цель работы	2
2. Вычислительная реализация задачи	3
2.1. Линейная аппроксимация	3
2.2. Квадратичная аппроксимация	4
2.3. Выбор лучшей аппроксимации	5
2.4. График функции и полученных аппроксимаций	6
3. Программная реализация задачи	7
3.1. Примеры кода	7
3.1.1. Линейная аппроксимация	7
3.1.2. Универсальная аппроксимация полиномом заданной степени	8
3.1.3. Показательная функция (пример линеаризации)	9
3.2. Пример работы программы	11
3.2.1. <code>stdout</code>	11
3.2.2. Графики (<code>pdf+png</code>)	13
4. Вывод	14

1. Цель работы

Найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов

2. Вычислительная реализация задачи

$$y = \frac{6x}{x^4 + 5}, x \in [0, 2], h = 0, 2$$

$$n = 11$$

x	0	0, 2	0, 4	0, 6	0, 8	1	1, 2	1, 4	1, 6	1, 8	2
y	0	0.24	0.48	0.70	0.89	1	1.02	0.95	0.83	0.7	0.57

Таблица 1. Таблица значений $f(x)$ для $x \in [0; 2]$ с шагом $h = 0, 2$

2.1. Линейная аппроксимация

$$SX = \sum x_i = 11$$

$$SXX = \sum x_i^2 = 15.4$$

$$SY = \sum y_i = 7.38$$

$$SXY = \sum x_i \cdot y_i \approx 8.654$$

$$S = S(a, b) = \sum \varepsilon_i^2 = \sum [\varphi(x_i) - y_i]^2 = \sum (ax_i + b - y_i) \rightarrow \min$$

Необх. условие сущ. минимума для ф. S :

$$\begin{cases} \frac{\delta S}{\delta a} = 0 \\ \frac{\delta S}{\delta b} = 0 \end{cases} \Leftrightarrow \begin{cases} 2 \sum (ax_i + b - y_i)x_i = 0 \\ 2 \sum (ax_i + b - y_i) = 0 \end{cases}$$

Упростим:

$$\begin{cases} a \sum x_i^2 + b \sum x_i = \sum x_i y_i \\ a \sum x_i + b n = \sum y_i \end{cases} \Leftrightarrow \begin{cases} aSXX + bSX = SXY \\ aSX + bn = SY \end{cases}$$

По правилу Крамера:

$$\Delta = SXX \cdot n - SX \cdot SX = 15.4 \cdot 11 - 11 \cdot 11 = 169.4 - 121 = 48.4$$

$$\Delta_1 = SXY \cdot n - SX \cdot SY = 8.654 \cdot 11 - 11 \cdot 7.38 = 95.194 - 81.18 = 14.014$$

$$\Delta_2 = SXX \cdot SY - SX \cdot SXY = 15.4 \cdot 7.38 - 11 \cdot 8.654 = 113.652 - 95.194 = 18.458$$

$$a = \frac{\Delta_1}{\Delta} \approx 0.29, \quad b = \frac{\Delta_2}{\Delta} \approx 0.38$$

$$\varphi(x) = 0.29x + 0.38$$

i	1	2	3	4	5	6	7	8	9	10	11
x_i	0	0, 2	0, 4	0, 6	0, 8	1	1, 2	1, 4	1, 6	1, 8	2
y_i	0	0.24	0.48	0.70	0.89	1	1.02	0.95	0.83	0.7	0.57
$\varphi(x_i)$	0.38	0.438	0.496	0.554	0.612	0.67	0.728	0.786	0.844	0.90	0.96
$\varphi(x_i)^2$	0.144	0.192	0.246	0.307	0.375	0.449	0.530	0.618	0.712	0.810	0.922

$$\sigma = \sqrt{\frac{\sum (\varphi(x_i) - y_i)^2}{n}} \approx 0.220$$

2.2. Квадратичная аппроксимация

$$\varphi(x) = a + bx + cx^2$$

$$SX = \sum x_i = 11.0$$

$$SXX = \sum x_i^2 = 15.4$$

$$SXXX = \sum x_i^3 = 24.2$$

$$SXXXX = \sum x_i^4 = 40.5328$$

$$SY = \sum y_i = 7.38$$

$$SXY = \sum x_i y_i = 8.654$$

$$SXXY = \sum x_i^2 y_i = 11.9116$$

Решаем систему (например, методом Крамера, аналогично с **линейной**)

$$\begin{cases} 11a + 11b + 15.4c = 7.38 \\ 11a + 15.4b + 24.2c = 8.654 \\ 15.4a + 24.2b + 40.5328c = 11.9116 \end{cases} \Rightarrow a \approx -0.042, \quad b \approx 1.701, \quad c \approx -0.706$$

$$\varphi(x) = -0.042 + 1.701x - 0.706x^2$$

i	1	2	3	4	5	6
x_i	0	0.2	0.4	0.6	0.8	1
y_i	0	0.24	0.48	0.70	0.89	1
$\varphi(x_i)$	-0.042	0.270	0.525	0.724	0.867	0.953
$(\varphi(x_i) - y_i)^2$	0.00176	0.00090	0.00203	0.00058	0.00053	0.00221

Таблица 3. Таблица значений $\varphi(x_i)$ и квадратов отклонений (часть 1: $i = 1...6$)

i	7	8	9	10	11
x_i	1.2	1.4	1.6	1.8	2
y_i	1.02	0.95	0.83	0.7	0.57
$\varphi(x_i)$	0.983	0.956	0.872	0.732	0.536
$(\varphi(x_i) - y_i)^2$	0.00137	0.00004	0.00176	0.00102	0.00116

Таблица 4. Таблица значений $\varphi(x_i)$ и квадратов отклонений (часть 2: $i = 7...11$)

Разделил таблицу, потому что не влезали числа

Сумма квадратов отклонений:

$$\sum (\varphi(x_i) - y_i)^2 \approx 0.01335$$

Среднеквадратичное отклонение:

$$\sigma = \sqrt{\frac{\sum (\varphi(x_i) - y_i)^2}{n}} \approx \sqrt{\frac{0.01335}{11}} \approx 0.035$$

2.3. Выбор лучшей аппроксимации

Вид аппроксимации σ

Линейная 0.220

Квадратичная 0.035

Выбираем квадратичную аппроксимацию, потому что её СКО меньше, чем у линейной. Итого, получаем:

$$\varphi(x) = -0.042 + 1.701x - 0.706x^2$$

2.4. График функции и полученных аппроксимаций

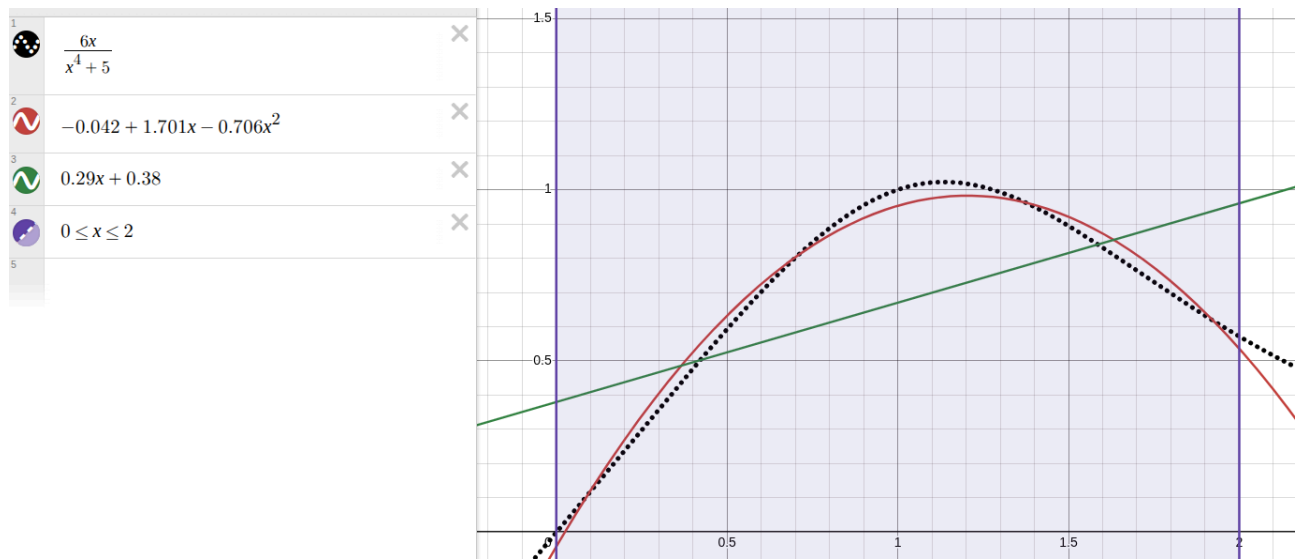


Рис. 1. График функции и аппроксимаций (красная – квадратичная, зеленая – линейная, черная – исходная)

3. Программная реализация задачи

Программа реализована как CLI-приложение на ЯП Python с использованием библиотек:

- `numpy` для матричных и векторных вычислений
- `matplotlib` для построения графиков

Полный код находится в репозитории по ссылке: https://github.com/xtern0o/vm_labs/blob/main/vm_lab4

3.1. Примеры кода

3.1.1. Линейная аппроксимация

```
"""Линейная аппроксимация:  $\phi(x) = ax + b$ """

import numpy as np
from ..base import ApproximationFunction

class LinearApproximation(ApproximationFunction):
    """Линейная аппроксимация:  $\phi(x) = ax + b$ """

    def fit(self) -> np.ndarray:
        """Найти коэффициенты  $a$  и  $b$ """
        sx = np.sum(self.x)
        sxx = np.sum(self.x ** 2)
        sy = np.sum(self.y)
        sxy = np.sum(self.x * self.y)

        delta = sxx * self.n - sx * sx

        if delta == 0:
            raise ValueError("Система вырождена, невозможно найти решение")

        delta1 = sxy * self.n - sx * sy
        delta2 = sxx * sy - sx * sxy

        a = delta1 / delta
        b = delta2 / delta

        self.coefficients = np.array([a, b])

        self.approximated_y = None
        self.errors = None
```



```

        return self.coefficients

    def evaluate(self, x_values: np.ndarray) -> np.ndarray:
        """Вычислить линейную аппроксимацию"""
        if self.coefficients is None:
            self.fit()

        a, b = self.coefficients
        return a * x_values + b

```

3.1.2. Универсальная аппроксимация полиномом заданной степени

"""Полиномиальная аппроксимация"""

```

import numpy as np
from ..base import ApproximationFunction
from ..solver import SeidelSolver

```

```

class PolynomialApproximation(ApproximationFunction):
    """Полиномиальная аппроксимация:  $\phi(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ """

    def __init__(self, x: np.ndarray, y: np.ndarray, degree: int):
        """
        Args:
            x: Массив значений x
            y: Массив значений y
            degree: Степень полинома
        """
        super().__init__(x, y)
        self.degree = degree
        self.solver = SeidelSolver(eps=1e-9, max_iter=5000)

    def fit(self) -> np.ndarray:
        """Найти коэффициенты полинома методом наименьших квадратов"""

        A = np.column_stack([self.x ** i for i in range(self.degree +
1)])
        AtA = A.T @ A
        Aty = A.T @ self.y

        # гаусс-зейдель
        try:

```

```

        result = self.solver.solve(AtA, Aty)
        self.coefficients = result.solution
    except RuntimeError as e:
        print(f"[Error]: {e}")
        print("    Используется прямой метод решения
(numpy.linalg.solve)...")
        self.coefficients = np.linalg.solve(AtA, Aty)

    self.approximated_y = None # ресет для перерасчета
    self.errors = None

    return self.coefficients

def evaluate(self, x_values: np.ndarray) -> np.ndarray:
    """Вычислить полином"""
    if self.coefficients is None:
        self.fit()

    result = np.zeros_like(x_values, dtype=float)
    for i, coef in enumerate(self.coefficients):
        result += coef * (x_values ** i)

    return result

```

3.1.3. Показательная функция (пример линеаризации)

```

"""Степенная аппроксимация:  $\phi(x) = a * x^{**b}$ """

import numpy as np
from ..base import ApproximationFunction

class PowerApproximation(ApproximationFunction):
    """Степенная аппроксимация:  $\phi(x) = a * x^{**b}$ """

    def fit(self) -> np.ndarray:
        """Найти коэффициенты a и b путем линеаризации"""
        # Линеаризация:  $\ln(\phi) = \ln(a) + b * \ln(x)$ 
        #  $Y = \ln(y)$ ,  $X = \ln(x)$ ,  $A = \ln(a)$ ,  $B = b$ 

        if np.any(self.x <= 0) or np.any(self.y <= 0):
            raise ValueError("Для степенной функции все x и y должны
быть > 0")

        X = np.log(self.x)
        Y = np.log(self.y)

```

```

# Применяем линейную регрессию к (X, Y)
sX = np.sum(X)
sXX = np.sum(X ** 2)
sY = np.sum(Y)
sXY = np.sum(X * Y)

delta = sXX * self.n - sX * sX

if delta == 0:
    raise ValueError("Система вырождена, невозможно найти
решение")

B = (sXY * self.n - sX * sY) / delta
A = (sXX * sY - sX * sXY) / delta

a = np.exp(A)
b = B

self.coefficients = np.array([a, b])
self.approximated_y = None
self.errors = None

return self.coefficients

def evaluate(self, x_values: np.ndarray) -> np.ndarray:
    """Вычислить  $\phi(x) = a * x^{**b}$ """
    if self.coefficients is None:
        self.fit()

    a, b = self.coefficients
    return a * np.power(x_values, b)

```

3.2. Пример работы программы

3.2.1. stdout

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Путь к JSON файлу (Enter для ввода вручную): ei2.json
Загружено 9 точек из ei2.json
Выполняется аппроксимация:
  1. Линейная:      f(x) = a*x + b
  2. Полином 2:      f(x) = a0 + a1*x + a2*x^2
  3. Полином 3:      f(x) = a0 + a1*x + a2*x^2 + a3*x^3
  4. Экспоненциальная: f(x) = a*exp(b*x)
  5. Логарифмическая: f(x) = a*ln(x) + b
  6. Степенная:      f(x) = a*x^b

[Error]: Метод Гаусса-Зейделя не сошелся за 5000 итераций. Последняя погрешность: 1.30e-04
Используется прямой метод решения (numpy.linalg.solve)...

ВЫБОР ЛУЧШЕЙ АППРОКСИМАЦИИ
-----
Лучшая функция: polynomial_3
S (сумма квадратов) = 2.880736e-03
sigma (сигма) = 1.789083e-02
R2 (детерминация) = 0.990228

ВЫВОД РЕЗУЛЬТАТОВ
-----
Путь для сохранения JSON (Enter для консоли):

=== РЕЗУЛЬТАТЫ АППРОКСИМАЦИИ ===

Количество точек: 9

Результаты для всех функций:
-----
Функция          S              sigma          R2
-----
exponential      2.998400e-01    1.825255e-01    -0.001710
linear           2.941983e-01    1.808002e-01    0.002041
logarithmic      2.747441e-01    1.747201e-01    0.068032
polynomial_2     9.143463e-03    3.187382e-02    0.968984
polynomial_3     2.880736e-03    1.789083e-02    0.990228    [ЛУЧШАЯ]
power            2.852319e-01    1.780237e-01    0.044673
```

Рис. 2. Общая информация об аппроксимации и выбор лучшей

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Детали по каждой функции:
-----

exponential:
  a = 0.73339345
  b = 0.04137635
  S = 2.998400e-01
  sigma = 1.825255e-01
  R2 = -0.001710

=====
Таблица значений
=====
i   x           y           phi(x)      eps_i
-----
1   0.400000    0.480000    0.745633    0.265633
2   0.600000    0.700000    0.751828    0.051828
3   0.800000    0.890000    0.758076    -0.131924
4   1.000000    1.000000    0.764375    -0.235625
5   1.200000    1.020000    0.770727    -0.249273
6   1.400000    0.950000    0.777131    -0.172869
7   1.600000    0.830000    0.783589    -0.046411
8   1.800000    0.700000    0.790100    0.090100
9   2.000000    0.570000    0.796666    0.226666
=====

linear:
  a = 0.01583333
  b = 0.77433333
  S = 2.941983e-01
  sigma = 1.808002e-01
  R2 = 0.002041
  r = 0.045177

=====
Таблица значений
=====
i   x           y           phi(x)      eps_i
-----
1   0.400000    0.480000    0.745633    0.265633
2   0.600000    0.700000    0.751828    0.051828
3   0.800000    0.890000    0.758076    -0.131924
4   1.000000    1.000000    0.764375    -0.235625
5   1.200000    1.020000    0.770727    -0.249273
6   1.400000    0.950000    0.777131    -0.172869
7   1.600000    0.830000    0.783589    -0.046411
8   1.800000    0.700000    0.790100    0.090100
9   2.000000    0.570000    0.796666    0.226666
=====
```

Рис. 3. Детали по каждой функции

3.2.2. Графики (pdf+png)

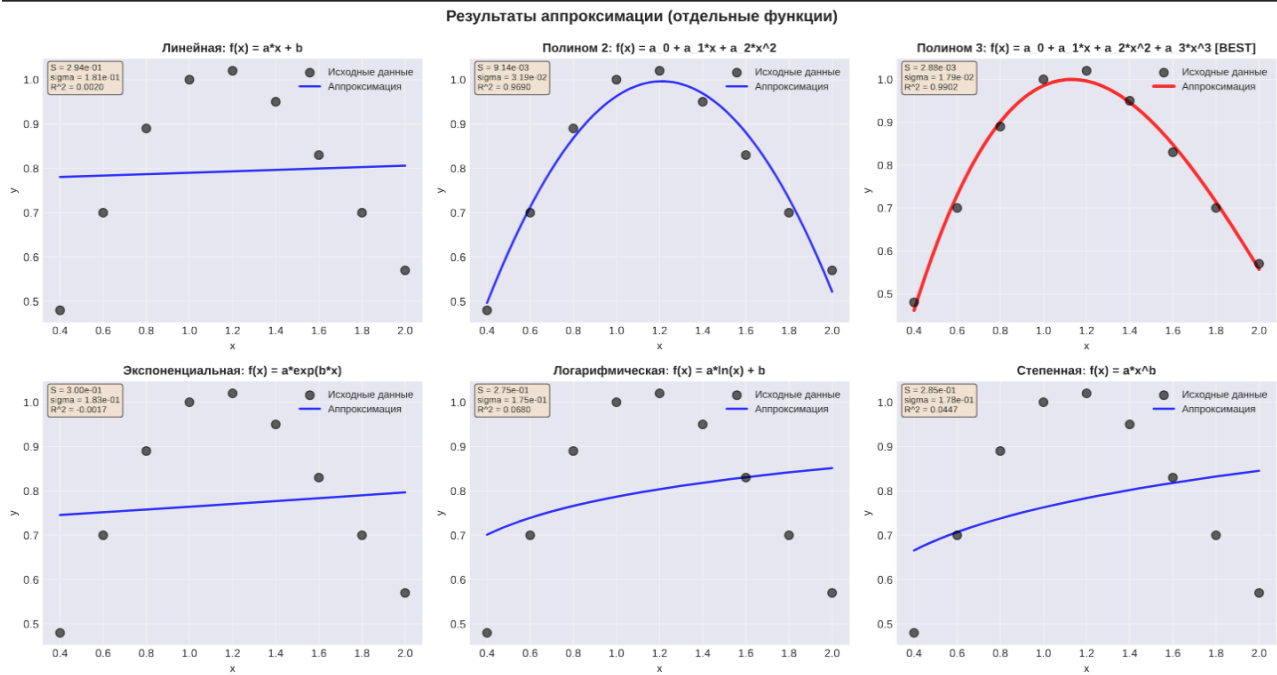


Рис. 4. Вывод графиков «каждый по отдельности»

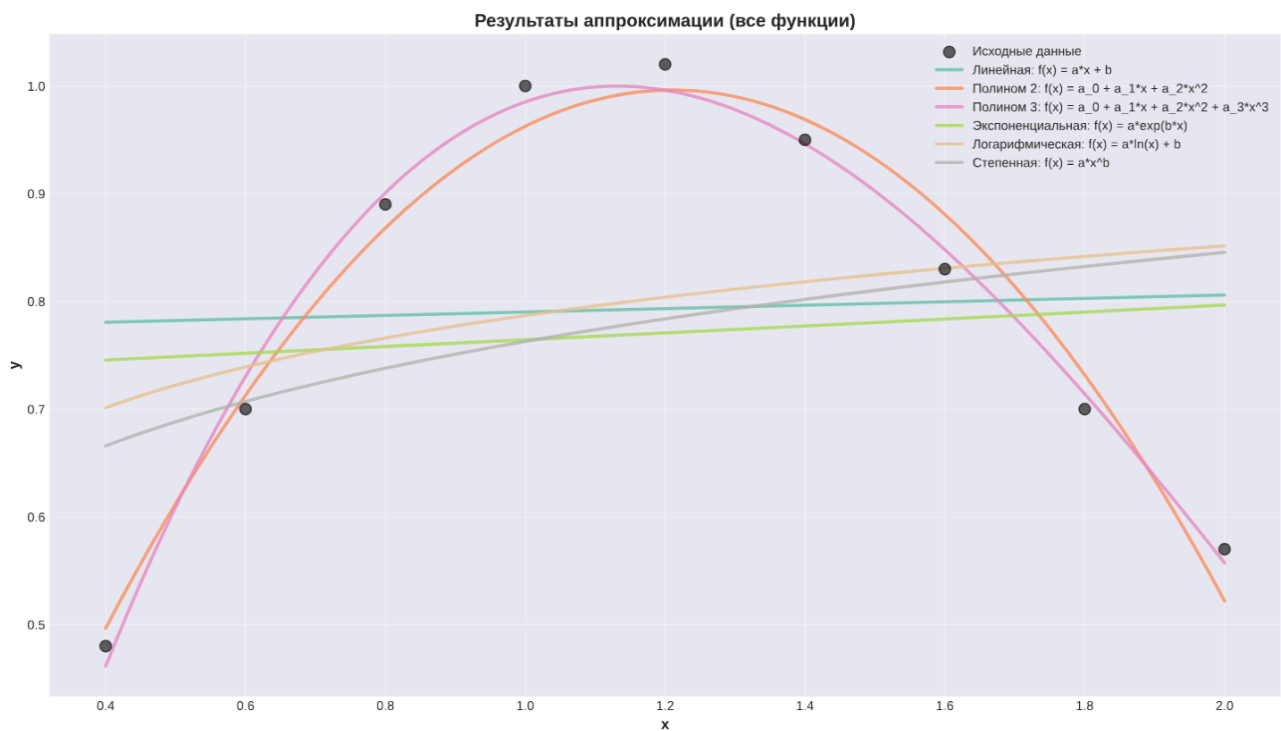


Рис. 5. Вывод графиков «все на одном рисунке»

4. Вывод

В ходе лабораторной работы я познакомился с основными способами аппроксимации: линейным, полиномиальным, экспоненциальным и логарифмическим. Также на основе аппроксимаций вычислял коэффициенты точности: СКО, R^2 и r_{xy} (последний не на основе аппроксимации, конечно). Написал программу, которую буду использовать на лабораторных работах по физике - это клёво.